

Punteros, arreglos y arreglos bidimensionales

Estructuras de Datos

Carlos A Delgado S

Pontificia Universidad Javeriana Cali

Agosto 2026

Objetivos de aprendizaje I

Al finalizar esta sesión, el estudiante podrá

- Operar direcciones con la aritmética de punteros y explicar la equivalencia entre $A[i]$ y $*(A + i)$.
- Recorrer un arreglo con un puntero y con un índice, y pasar de una forma a la otra.
- Explicar cómo se guarda una matriz en memoria y calcular la dirección de $M[i][j]$.
- Reservar, recorrer y liberar una matriz cuyo tamaño se conoce en ejecución.

Objetivos de aprendizaje II

Objetivos de Aprendizaje del curso involucrados

- OA1: apropiar conocimientos de computación mediante el diseño e implementación de soluciones a problemas pequeños.
- OA2: diseñar e implementar soluciones usando las herramientas del lenguaje.

Referencia bibliográfica

R. Thareja, *Data Structures Using C*, Oxford University Press, 2018. Capítulo 6 (aritmética de punteros, punteros y arreglos) y capítulo 3 (*Arrays*, incluidas las secciones de arreglos bidimensionales).

Contenido I

- 1 Lo que dejó el manejo de memoria
- 2 La aritmética de las direcciones
- 3 El arreglo es una dirección
- 4 La tabla en una sola calle
- 5 La matriz del tamaño justo
- 6 Ejercicios
- 7 Ejercicio de cierre
- 8 Resumen y referencias
- 9 Para practicar en casa

Contenido I

1 Lo que dejó el manejo de memoria

2 La aritmética de las direcciones

3 El arreglo es una dirección

4 La tabla en una sola calle

5 La matriz del tamaño justo

6 Ejercicios

7 Ejercicio de cierre

8 Resumen y referencias

9 Para practicar en casa

Lo que dejó el manejo de memoria I

int 4 bytes = 32 bits char = 1 byte = 8 bits
long 8 bytes = 64 bits

- La memoria es una calle de celdas numeradas; cada variable ocupa las suyas y $\&v$ es su dirección.
- Un puntero guarda una dirección; $*p$ usa la celda apuntada.
- malloc reserva en el montículo y se verifica con NULL; free devuelve; después de free, el puntero a NULL.
- Quedó pendiente una pregunta del mapa de direcciones: si A empieza en 2000, ¿por qué $A[3]$ vive en 2012?

Hoy los punteros y los arreglos resultan ser la misma historia, y la historia se extiende a las tablas.

Contenido I

1 Lo que dejó el manejo de memoria

2 La aritmética de las direcciones

3 El arreglo es una dirección

4 La tabla en una sola calle

5 La matriz del tamaño justo

6 Ejercicios

7 Ejercicio de cierre

8 Resumen y referencias

9 Para practicar en casa

¿Cuánto vale $p + 1$? I

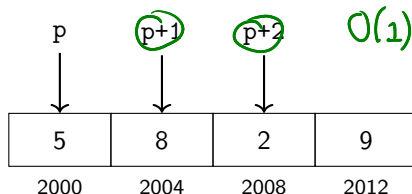
p es un `int *` y su valor es 2000.

En parejas: ¿cuánto vale $p + 1$? ¿2001 o 2004?

¿Cuánto vale $p + 1$?: respuesta I

Aritmética de punteros (Thareja, cap. 6)

Sumarle k a un puntero lo avanza k casillas del tipo apuntado, no k bytes: para un `int *`, $p + k$ vale $p + 4k$.



- $p + 1$ vale 2004: la casilla siguiente, no el byte siguiente. El compilador multiplica por `sizeof(int)` por nosotros.
- Con un `double *` en 3000, $q + 2$ vale 3016: cada casilla mide 8.

¿Cuánto vale $p + 1$?: respuesta II

- La resta también funciona: entre dos punteros al mismo arreglo, $q - p$ cuenta casillas.

Contenido I

- 1 Lo que dejó el manejo de memoria
- 2 La aritmética de las direcciones
- 3 El arreglo es una dirección**
- 4 La tabla en una sola calle
- 5 La matriz del tamaño justo
- 6 Ejercicios
- 7 Ejercicio de cierre
- 8 Resumen y referencias
- 9 Para practicar en casa

El nombre del arreglo I

Punteros y arreglos (Thareja, cap. 6)

En una expresión, el nombre de un arreglo vale la dirección de su primera casilla: A equivale a $\&A[0]$. Y el corchete es aritmética disfrazada:

$$A[i] \equiv *(A + i).$$

La pregunta pendiente queda respondida: $A[3]$ es $*(A + 3)$, la casilla que está 3 lugares después de 2000, es decir $2000 + 3 \cdot 4 = 2012$.

La máquina está de acuerdo I

```
int A[4] = {5, 8, 2, 9};
int i;

printf("A vale %p\n", (void *) A);
i = 0;
while (i < 4) {
    printf("A[%d] = %d    *(A+%d) = %d    A+%d = %p\n",
          i, A[i], i, *(A + i), i, (void *) (A + i));
    i = i + 1;
}
```

Una corrida:

```
A vale 0x7ffdb78df870
A[0] = 5    *(A+0) = 5    A+0 = 0x7ffdb78df870
A[1] = 8    *(A+1) = 8    A+1 = 0x7ffdb78df874
A[2] = 2    *(A+2) = 2    A+2 = 0x7ffdb78df878
A[3] = 9    *(A+3) = 9    A+3 = 0x7ffdb78df87c
```

La máquina está de acuerdo II

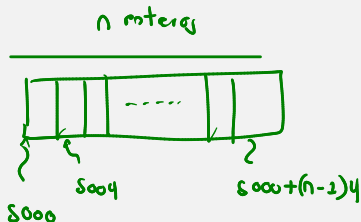
$A[i]$ y $*(A + i)$ coinciden casilla por casilla, y las direcciones suben de 4 en 4 desde A .

Recorrer caminando el puntero I

El mismo recorrido de siempre, escrito dos veces:

```
/* con indice */  
suma = 0;  
i = 0;  
while (i < n) {  
    { suma = suma + A[i];  
      i = i + 1;  
    }  
}
```

```
/* con puntero */  
suma = 0;  
p = A;  
while (p < A + n) {  
    { suma = suma + *p;  
      p = p + 1;  
    }  
}
```



Recorrer caminando el puntero II

- El índice cuenta casillas desde 0; el puntero se para en cada casilla. $p < A + n$ es el mismo tope que $i < n$.
- Las dos versiones cuestan lo mismo: $O(n)$. Elegir una es cuestión de claridad, y en el curso se usan ambas.

El arreglo viaja como dirección I

Empaquetado en una función, el recorrido con puntero queda así:

```
/* Suma los n enteros que empiezan en la direccion inicio */
int suma(int *inicio, int n) {
    int total = 0;
    int *p = inicio; Copia del puntero

    { while (p < inicio + n) {
        total = total + *p;
        p = p + 1;
    }
    return total;
}

int main(void) {
    int A[5] = {4, 7, 1, 3, 5};

    printf("La suma es %d\n", suma(A, 5));
    return 0;
}
```

El arreglo viaja como dirección II

```
}
```

```
La suma es 20
```

En la llamada `suma(A, 5)`, lo que viaja es la dirección de `A[0]`, no una copia del arreglo. Por eso la función necesita que le digan `n`: con la dirección sola no se sabe dónde termina. Este viaje de direcciones se vuelve tema propio en el paso de parámetros por referencia.

¿Qué imprime? I

En parejas, con una traza de direcciones inventadas:

```
int A[5] = {2, 4, 6, 8, 10};  
int *p = A + 1;  
printf("%d\n", *p + *(p + 2));
```

4 8 12

¿Qué imprime?: respuesta I

Con A en 2000:

Expresión	Dirección	Valor
$p = A + 1$	2004	—
$*p$	2004	4
$p + 2$	2012	—
$*(p + 2)$	2012	8

$$*p + *(p + 2) = 4 + 8 = 12.$$

El puntero puede arrancar en mitad del arreglo: p es $\&A[1]$, y desde ahí $p + 2$ es $\&A[3]$.

La casa de al lado I

A tiene 4 casillas: A[0] a A[3]. ¿Y si se lee A[4]?

```
int A[4] = {1, 2, 3, 4};  
  
printf("A[3] = %d\n", A[3]);  
printf("A[4] = %d\n", A[4]);    /* la casa de al lado */
```

```
A[3] = 4  
A[4] = 0
```

La casa de al lado II

El lenguaje no revisa los límites

$A[4]$ es $*(A + 4)$: una dirección válida como cualquiera, la casa de al lado. Esta corrida imprimió 0; otra puede imprimir cualquier cosa, y una *escritura* ahí corrompe la variable vecina. El compilador guardó silencio: a diferencia del puntero sin inicializar del manejo de memoria, aquí no hay aviso. El límite lo cuida el programa, con la condición del ciclo.

Contenido I

- 1 Lo que dejó el manejo de memoria
- 2 La aritmética de las direcciones
- 3 El arreglo es una dirección
- 4 La tabla en una sola calle**
- 5 La matriz del tamaño justo
- 6 Ejercicios
- 7 Ejercicio de cierre
- 8 Resumen y referencias
- 9 Para practicar en casa

La planilla de notas I

Tres estudiantes, cuatro tareas:

	T1	T2	T3	T4
Ana	10	11	12	13
Benito	20	21	22	23
Clara	30	31	32	33

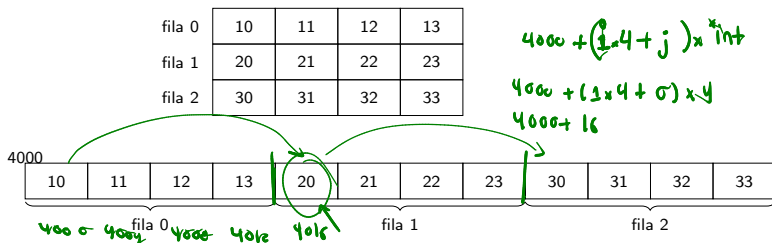
Los datos son una tabla: fila el estudiante, columna la tarea. Pero la memoria es una sola calle de celdas consecutivas.

¿Cómo se guarda una tabla en una calle?

El arreglo bidimensional I

Definición (Thareja, cap. 3, *Two-dimensional Arrays*)

`int M[3][4]` declara un arreglo de 3 filas y 4 columnas; `M[i][j]` es la casilla de la fila `i` y la columna `j`. En memoria, las filas se guardan una tras otra, sin huecos (orden por filas, *row-major*).



La tabla es una forma de leer la calle: el compilador guarda las 12 celdas seguidas y hace las cuentas.

La cuenta del compilador I

Si M tiene m columnas y empieza en la dirección b :

dirección de $M[i][j] = b + (i \cdot m + j) \cdot \text{sizeof}(\text{int})$.

Saltar i filas completas ($i \cdot m$ casillas) y avanzar j dentro de la fila.

En parejas: `int M[3][4]` empieza en 4000. ¿Dónde vive $M[2][1]$?

4000

$$\begin{aligned} M[3][4] &= 4000 + (3 \times m + 4) \times 4 \\ &= 4000 + 12m + 16 \end{aligned}$$

$$\begin{aligned} M[2][1] &= 4000 + (2 \times m + 1) \times 4 \\ &= 4000 + 8m + 4 \end{aligned}$$

La cuenta del compilador: respuesta I

$$4000 + (2 \cdot 4 + 1) \cdot 4 = 4000 + 36 = 4036.$$

Y la máquina está de acuerdo. Direcciones reales de la planilla:

	0	1	2	3
0	c20	c24	c28	c2c
1	c30	c34	c38	c3c
2	c40	c44	c48	c4c

- De $M[0][3]$ a $M[1][0]$ hay 4 bytes: la fila 1 empieza donde termina la fila 0, sin huecos.
- De fila a fila hay 16 bytes: 4 casillas de 4.
- Igual que con $A[i]$: llegar a $M[i][j]$ es una cuenta, no un recorrido. Cuesta $O(1)$.

Recorrer la planilla I

Un ciclo por dimensión: el externo elige la fila, el interno la recorre.

```
int M[3][4] = {  
    {10, 11, 12, 13},  
    {20, 21, 22, 23},  
    {30, 31, 32, 33}  
};
```

```
int i, j, total;
```

```
i = 0;
```

```
while (i < 3) {
```

```
    total = 0;
```

```
    j = 0;
```

```
    while (j < 4) {
```

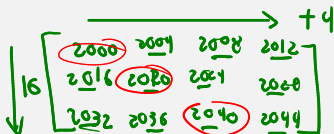
```
        total = total + M[i][j];
```

```
        j = j + 1;
```

```
    }
```

```
    printf("Fila %d: suma %d\n", i, total);
```

```
    i = i + 1;
```



$\sim O(1)$

Recorrer la planilla II

```
}
```

```
Fila 0: suma 46
```

```
Fila 1: suma 86
```

```
Fila 2: suma 126
```

Los ciclos anidados independientes de la semana de complejidad:
 3×4 vueltas del cuerpo, $O(n \cdot m)$ en general.

↳ $O(n^2)$

Ahora usted I

En parejas: cambie el recorrido para sumar la *columna* 2 de la planilla (la tarea T3). ¿Cuántas vueltas da el ciclo?

Ahora usted: respuesta I

Basta un ciclo: la columna queda fija y la fila avanza.

```
total = 0;
i = 0;
while (i < 3) {
    total = total + M[i][2];
    i = i + 1;
}
```

3x4

2000

2008

1

0

2024

1

2040

2

1

Tres vueltas, y $total = 12 + 22 + 32 = 66$.

En la calle, esas tres casillas no son vecinas: viven en 4008, 4024 y 4040, a 16 bytes una de otra. Recorrer por columnas salta por la memoria; recorrer por filas la camina en orden. La diferencia costará tiempo real cuando las matrices crezcan.

Contenido I

- 1 Lo que dejó el manejo de memoria
- 2 La aritmética de las direcciones
- 3 El arreglo es una dirección
- 4 La tabla en una sola calle
- 5 La matriz del tamaño justo**
- 6 Ejercicios
- 7 Ejercicio de cierre
- 8 Resumen y referencias
- 9 Para practicar en casa

Filas y columnas que decide el problema I

La planilla real no es de 3×4 : los estudiantes y las tareas se conocen al ejecutar. En el manejo de memoria la solución fue `malloc`, y sirve igual aquí, con una idea prestada del compilador:

Una matriz en un solo bloque

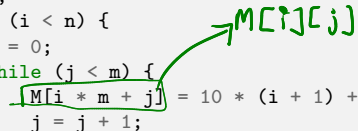
Reservar $n \cdot m$ casillas seguidas y hacer a mano la cuenta del *row-major*: la casilla (i, j) es $M[i * m + j]$.

```
M = malloc(n * m * sizeof(int));
```

El bloque queda en el montículo; n , m y el puntero M , en la pila. Un solo `malloc`, un solo `NULL` que revisar y un solo `free`.

La matriz del tamaño justo: el programa I

```
printf("Filas y columnas? ");
if (scanf("%d %d", &n, &m) != 2 || n <= 0 || m <= 0) {
    printf("Tamano invalido\n");
    estado = 1;
} else {
    M = malloc(n * m * sizeof(int));
    if (M == NULL) {
        printf("No hay memoria disponible\n");
        estado = 1;
    } else {
        i = 0;
        while (i < n) {
            j = 0;
            while (j < m) {
                M[i * m + j] = 10 * (i + 1) + j;
                j = j + 1;
            }
            i = i + 1;
        }
        /* ... imprimir igual, con M[i * m + j] ... */
        free(M);
    }
}
```



La matriz del tamaño justo: el programa II

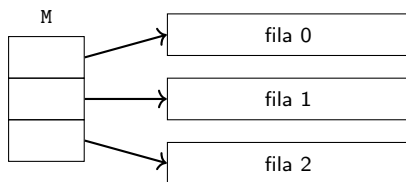
```
        M = NULL;  
    }  
}
```

Una corrida con 3×4 reproduce la planilla:

```
Filas y columnas? 3 4  
10 11 12 13  
20 21 22 23  
30 31 32 33
```

La otra forma de armar una matriz I

Existe una variante: un arreglo de punteros, donde cada fila pide su propio bloque.



- Permite escribir $M[i][j]$ y filas de largos distintos, a cambio de $n + 1$ `malloc` con sus $n + 1$ `free` y sus verificaciones.
- En el curso usamos el bloque único: una reserva, una liberación, y la cuenta del *row-major* a la vista. La variante reaparecerá cuando las estructuras enlazadas nos den las herramientas para manejarla con comodidad.

Contenido I

- 1 Lo que dejó el manejo de memoria
- 2 La aritmética de las direcciones
- 3 El arreglo es una dirección
- 4 La tabla en una sola calle
- 5 La matriz del tamaño justo
- 6 Ejercicios**
- 7 Ejercicio de cierre
- 8 Resumen y referencias
- 9 Para practicar en casa

Ejercicio I

En parejas: escriba el programa que encuentra el mayor valor de la planilla y *en qué fila y columna* está.

```
int M[3][4] = {  
    {3, 9, 1, 4},  
    {7, 2, 8, 5},  
    {6, 0, 2, 1}  
};
```

Antes de escribir: ¿qué variables hay que recordar mientras se recorre?

Ejercicio: respuesta I

El mayor visto hasta ahora, y su posición; los tres se actualizan juntos.

```
int mayor = M[0][0];
int fila = 0;
int columna = 0;

i = 0;
while (i < 3) {
    j = 0;
    while (j < 4) {
        if (M[i][j] > mayor) {
            mayor = M[i][j];
            fila = i;
            columna = j;
        }
        j = j + 1;
    }
    i = i + 1;
```

Ejercicio: respuesta II

```
}  
printf("El mayor es %d, en la fila %d y la columna %d\n",  
      mayor, fila, columna);
```

```
El mayor es 9, en la fila 0 y la columna 1
```

El esqueleto de buscar el mayor, de la semana de complejidad, ahora con dos coordenadas por recordar.

Contenido I

1 Lo que dejó el manejo de memoria

2 La aritmética de las direcciones

3 El arreglo es una dirección

4 La tabla en una sola calle

5 La matriz del tamaño justo

6 Ejercicios

7 Ejercicio de cierre

8 Resumen y referencias

9 Para practicar en casa

Ejercicio de cierre I

Escriba un programa completo que:

- 1 Lea n y m , y reserve una matriz de $n \times m$ enteros en un solo bloque.
- 2 Lea sus $n \cdot m$ valores.
- 3 Encuentre la fila de mayor suma y la imprima con su índice.
- 4 Devuelva la memoria.

Con las verificaciones y las tres reglas del manejo de memoria.

Contenido I

- 1 Lo que dejó el manejo de memoria
- 2 La aritmética de las direcciones
- 3 El arreglo es una dirección
- 4 La tabla en una sola calle
- 5 La matriz del tamaño justo
- 6 Ejercicios
- 7 Ejercicio de cierre
- 8 Resumen y referencias**
- 9 Para practicar en casa

Resumen I

Lo que queda de la sesión

- Sumar k a un puntero avanza k casillas del tipo; A vale $\&A[0]$ y $A[i]$ es $*(A + i)$.
- Un arreglo se recorre con índice o caminando un puntero; el costo es el mismo.
- El lenguaje no revisa límites: $A[n]$ es la casa de al lado y el ciclo es quien cuida la frontera.
- Una matriz vive fila tras fila; $M[i][j]$ está en $b + (i \cdot m + j) \cdot 4$ y llegar cuesta $O(1)$.
- La matriz de tamaño justo: un bloque de $n \cdot m$ con $M[i * m + j]$, su NULL y su free.

Lo que sigue: el viaje de las direcciones a las funciones (paso de parámetros por referencia) y el repaso de POO.

Referencias I

-  R. Thareja. *Data Structures Using C*. Oxford University Press, 2018. Capítulo 3 (*Arrays*) y capítulo 6 (aritmética de punteros, punteros y arreglos, asignación dinámica).
-  N. Kalicharan. *Data Structures in C*. 2008. Arreglos, punteros y memoria dinámica.

Contenido I

1 Lo que dejó el manejo de memoria

2 La aritmética de las direcciones

3 El arreglo es una dirección

4 La tabla en una sola calle

5 La matriz del tamaño justo

6 Ejercicios

7 Ejercicio de cierre

8 Resumen y referencias

9 Para practicar en casa

Propuesto 1 I

Sin correrlo: ¿qué imprime? Dibuje las 6 casillas y los dos punteros, y siga las vueltas en una tabla.

```
int A[6] = {1, 3, 5, 7, 9, 11};
int *p = A;
int *q = A + 5;
int suma = 0;

while (p < q) {
    suma = suma + *p + *q;
    p = p + 1;
    q = q - 1;
}
printf("suma = %d\n", suma);
```

Propuesto 2 I

Escriba una función que invierta un arreglo *en su lugar* (sin arreglo auxiliar), usando dos punteros que se acercan como los del propuesto 1. Pruébela con largos par e impar.

```
void invertir(int *inicio, int n);
```

Propuesto 3 I

`int M[4][4]` guarda una matriz cuadrada. Escriba el recorrido que suma su diagonal (`M[0][0]`, `M[1][1]`, ...) y responda: si `M` empieza en 5000, ¿en qué direcciones viven las casillas de la diagonal?

Propuesto 4 I

Sin correrlo: ¿qué imprime?

```
int A[5] = {10, 20, 30, 40, 50};  
int *p = A + 4;  
int *q = A + 1;  
  
printf("%d %ld\n", *p - *q, (long) (p - q));
```

¿Qué mide cada una de las dos restas?

Pistas de los propuestos I

- **Propuesto 1:** los punteros avanzan uno hacia el otro y cada vuelta aporta una pareja extremo-extremo: $1 + 11$, luego $3 + 9$, luego $5 + 7$. Tras la tercera vuelta se cruzan y el ciclo termina: $\text{suma} = 36$, la suma de Gauss contada desde las puntas.
- **Propuesto 2:** un puntero en cada extremo; intercambiar $*p$ y $*q$ con una variable auxiliar y acercarlos. El ciclo es `while (p < q)`: con largo impar, la casilla del centro no necesita moverse.
- **Propuesto 3:** la diagonal es $M[i][i]$: un solo ciclo. Con la fórmula, la casilla (i, i) vive en $5000 + (4i + i) \cdot 4 = 5000 + 20i$: direcciones 5000, 5020, 5040, 5060, a paso constante.
- **Propuesto 4:** $*p - *q$ resta *valores*: $50 - 20 = 30$. $p - q$ resta *direcciones* y da casillas: 3, no 12 bytes. Salida: 30 3.